This is the peer reviewed version of the following article: A. Marinho, L. Murta, C. Werner, V. Braganholo, S. M. S. da Cruz, E. Ogasawara, M. Mattoso. ProvManager: a provenance management system for scientific workflows. Concurrency and Computation: Practice and Experience 24(13): 1513–1530 (2012), which has been published in final form at https://doi.org/10.1002/cpe.1870. This article may be used for non-commercial purposes in accordance with Wiley Terms and Conditions for Use of Self-Archived Versions.

ProvManager: A Provenance Management System for Scientific Workflows

Anderson Marinho[1], Leonardo Murta[2], Cláudia Werner[1], Vanessa Braganholo[2], Sérgio Manuel Serra da Cruz[1,4], Eduardo Ogasawara[1,3] and Marta Mattoso[1]

[1]COPPE – Federal University of Rio de Janeiro, Brazil
[2]Computing Institute – Fluminense Federal University, Brazil
[3]Federal Center of Technological Education Celso Suckow da Fonseca, Brazil
[4]ICE - Federal Rural University of Rio de Janeiro, Brazil

{andymarinho,werner,serra,ogasawara,marta}@cos.ufrj.br,
{leomurta,vanessa}@ic.uff.br

SUMMARY

Running scientific workflows in distributed and heterogeneous environments has been motivating approaches for provenance management that are loosely coupled to the workflow execution engine. This kind of approach is interesting because it allows both storage and access to provenance data in a homogeneous way, even in an environment where different workflow management systems work together. However, current approaches overload scientists with many ad-hoc tasks, such as script adaptations and implementations of extra functionalities to provide provenance independence. This paper proposes ProvManager, a provenance management approach that eases the gathering, storage, and analysis of provenance information in a distributed and heterogeneous environment scenario, without putting the burden of adaptations on the scientist. ProvManager leverages the provenance management at experiment level by integrating different workflow executions from multiple workflow management systems.

KEY WORDS: provenance management, scientific experiment, scientific workflow.

1. INTRODUCTION

High Performance Computing (HPC) and scientific workflows have evolved as important resources to scientists, assisting in simulation and interpretation of experimental data. As a result, they provide important leads for further experimental research.

The workflow concept is used to support scientific experiments based on simulations, allowing a structured composition of programs as a sequence of activities that aims at a specific result [1]. Afterwards, scientific workflow management systems (SWfMS) were envisioned to control the orchestration and execution of these so-called scientific workflows. SWfMS not only make the steps of the workflow execution explicit, but also enable an automatic collection of workflow data products and their

provenance information [2].

Provenance data provide historical information about data manipulated in a workflow [3]. This historical information expresses how data products were generated, showing their transformation processes from primary input and intermediary data. The management of this kind of information is extremely important, since it provides scientists with a variety of data analysis applications. For instance, from provenance information it is possible to verify data quality of generated data products, because one can look at the ancestral data to check reliability. Other examples are [4]: possibility to audit trails to verify what resources are being used; data derivation capability; experiment documentation; responsibility attribution; among other applications. Provenance can be categorized into two main kinds [3]: prospective and retrospective. The prospective provenance captures a workflow specification as a recipe for future data derivation. The retrospective provenance captures past workflow execution and data derivation information.

In order to benefit scientists, provenance data have to be modeled, gathered, and stored for further queries. Provenance management is an open issue that is being addressed by several forums worldwide, including the Provenance Challenge at the International Provenance and Annotation Workshop (IPAW)[1]. One of the open problems relates to which provenance data should be gathered and how they can be collected. Provenance gathering becomes more complex when the experiment is executed at distributed and heterogeneous environments, such as clusters, P2P, grids, clouds, and several SWfMS.

One can foresee several scenarios of experiment execution in a distributed and heterogeneous environment. Often, a single experiment is executed by more than one workflow. The need for breaking a conceptual experiment in two or more separate workflow executions can occur due to several reasons. One reason is the presence of long manual activities along the experiment. To avoid a long break point in the workflow, it is often broken in two separate executions. Also a part of the experiment may need to be executed in a SWfMS that provides parallelism, while another part of the experiment may need to be executed in a system that supports result visualization. In this case, each SWfMS may manage provenance information in a decentralized and isolated way, meaning that each system considers provenance in a specific granularity, stores the information using a specific model, or even worse, some SWfMS may not provide a provenance solution at all. In situations like that, experiments would benefit from a homogeneous management of prospective and retrospective provenance.

The Open Provenance Model (OPM) [5] defines a generic representation for provenance data. It is an initiative towards a homogeneous model to provide interoperability between workflows from different SWfMS. However, this is only part of the solution. Even if all SWfMS are OPM compliant, there is still need to gather compatible provenance data and provide an integrated control of the distributed provenance data with query support.

A solution to this heterogeneity and distribution problems is to transfer the responsibility of provenance management to an independent provenance system. This system would be responsible for allowing modeling, capturing, storage, and querying provenance for the whole experiment.

[1] http://www.ipaw.info

Several works [6]-[9] share the idea of a provenance management system that is independent of a workflow system technology. However, the main difficulty of being SWfMS agnostic is that the SWfMS and the provenance management system need to communicate to exchange information. In order to make this communication possible, some researches [7],[10],[11] propose a series of manual activity adaptations over the workflow specification, but this solution introduces overhead to scientists. We claim that scientists should not have this computational burden. Besides, some workflow activities used by scientists are from third parties, which make their adaptation even more complex. In fact, in many cases these activities cannot be altered, at least not in a direct manner.

For this reason, the main goal of this paper is to introduce a novel approach for dealing with integrated provenance management in distributed and heterogeneous environments. In previous work [12], we have proposed a strategy for gathering provenance information in such environment. This strategy is independent of workflow system technology and addresses some of the previously discussed problems. We have evolved this strategy into a provenance management system named ProvManager. The main contribution of ProvManager, as described in this paper, is to allow scientists to focus on the essence of their experiments and make use of the best technologies to enact their workflows. ProvManager is able to transparently gather and store the provenance collected from different SWfMS, translating it to an integrated provenance model that represents the experiment as a whole. As a result, scientists are able to perform provenance queries over the experiment even if it is composed of multiple workflows and enacted on different SWfMS. This is possible due to the use of a knowledge base driven by an inference engine that serves as a provenance repository.

This paper is organized as follows. Section 2 presents ProvManager, discussing its purpose and characteristics. Section 3 discusses the architecture of ProvManager, describing its main components. Section 4 presents ProvManager in action, illustrating its operation. An evaluation study with ProvManager users is described in Section 5. Section 6 discusses related work. Finally, Section 7 concludes the paper.

2. PROVMANAGER

The ProvManager approach manages provenance in distributed and heterogeneous environments. As discussed before, in such scenarios, an integrated analysis of the experiment provenance is complex, since several SWfMS are used and the provenance information is generated and stored in a heterogeneous and decentralized way. This section describes ProvManager in terms of its provenance gathering strategy, and provides an overview of its operation.

2.1 Provenance gathering strategy

According to Freire et al. [3], Simmhan, Plale, and Gannon [4] and Cruz, Campos, and Mattoso [13], there are different kinds of provenance and also distinct kinds of provenance gathering mechanisms. In a broad sense, the provenance gathering mechanisms can be classified into three semantic levels: operating system (OS), workflow and activity.

Mechanisms that work at OS level gather provenance information using OS tools (*e.g.*, file-system and system call tracer). They are independent SWfMS, but can only

collect retrospective provenance, usually at a fine grain. At the workflow level, a SWfMS is responsible for gathering all the provenance information. The advantage is to gather both prospective and retrospective provenance, but the gathering mechanism is bound to a specific SWfMS and it is difficult to use the same mechanism in other systems. Finally, at the activity level, each workflow activity is responsible for gathering its own provenance information. The advantages are the SWfMS independency and the ease of implementation, just like in the OS level. However, mechanisms working at this level usually gathers retrospective provenance only, and demands extra effort to the scientists to adapt workflow activities.

ProvManager adopts the gathering of provenance at the activity level, since it provides SWfMS independency and allows collection of more precise and complete provenance data when compared to the OS level strategy. To mitigate the limitations of the activity level strategy (*i.e.*, the need of manual adaptation of the workflow activities and the lack of prospective provenance gathering), ProvManager introduces two new features: (i) a mechanism to automatically adapt the workflow activities, and (ii) a mechanism to collect prospective provenance from the workflow specification during the automatic workflow adaptation.

2.2 An Overview of ProvManager Operation

The ProvManager operation can be divided into three main phases: (a) provenance mechanism configuration, (b) provenance gathering, and (c) provenance analysis. Figure 1 illustrates the operation phases.

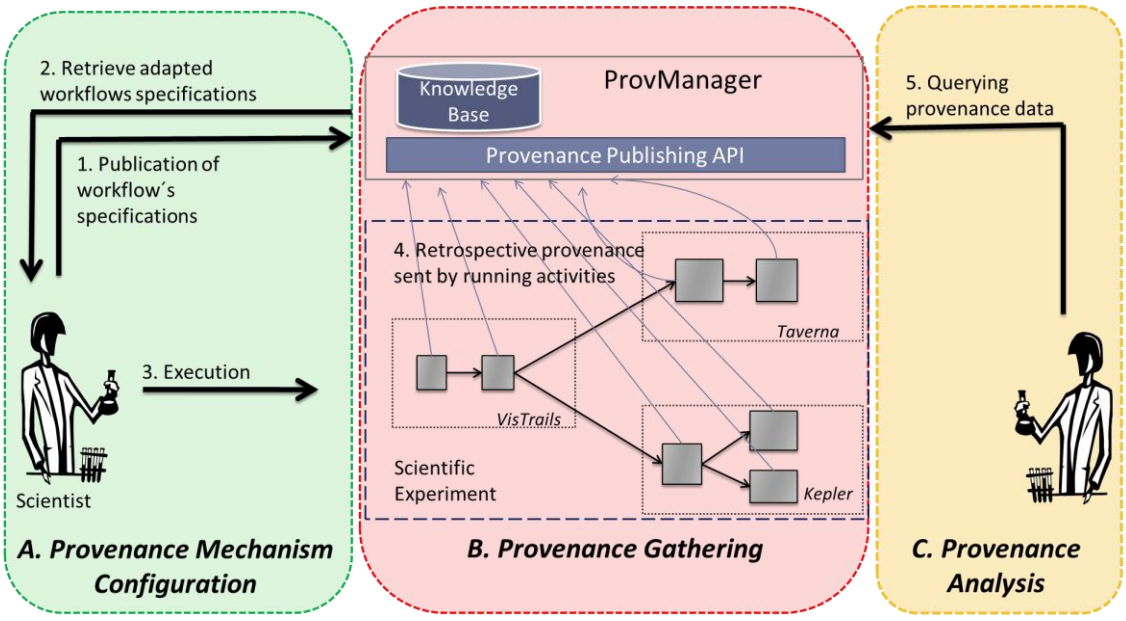


Figure 1. ProvManager in operation

At the configuration phase, when the workflows are composed in the SWfMS and ready to be executed, the scientist should first provide these workflow specifications to ProvManager (step 1 of Figure 1). These workflow specifications are automatically interpreted by ProvManager to extract the prospective provenience. Moreover, ProvManager also adapts these workflow specifications, including special activities that

are responsible for gathering retrospective provenance during execution. This process of adding provenance components into the workflow is called instrumentation. These instrumented workflow specifications are then returned to the scientist (step 2 of Figure 1). They can now be put into execution in the SWfMS (step 3 of Figure 1). During the execution of workflows, the retrospective provenance data are gathered by the before mentioned special activities instrumented in the workflows and published in the ProvManager repository (step 4 of Figure 1). At this moment, ProvManager binds the previously collected prospective provenance data with the just collected retrospective provenance data, being OPM compliant. Finally, at the provenance analysis phase, scientists may use ProvManager to query the experiment provenance data in an integrated fashion (step 5 of Figure 1). As discussed before, querying over the ProvManager repository is boosted by an inference engine that allows reasoning over the collected provenance data.

3. PROVMANAGER ARCHITECTURE

The architecture of ProvManager is illustrated in Figure 2. The components of the architecture are organized in layers, which are defined as follows.

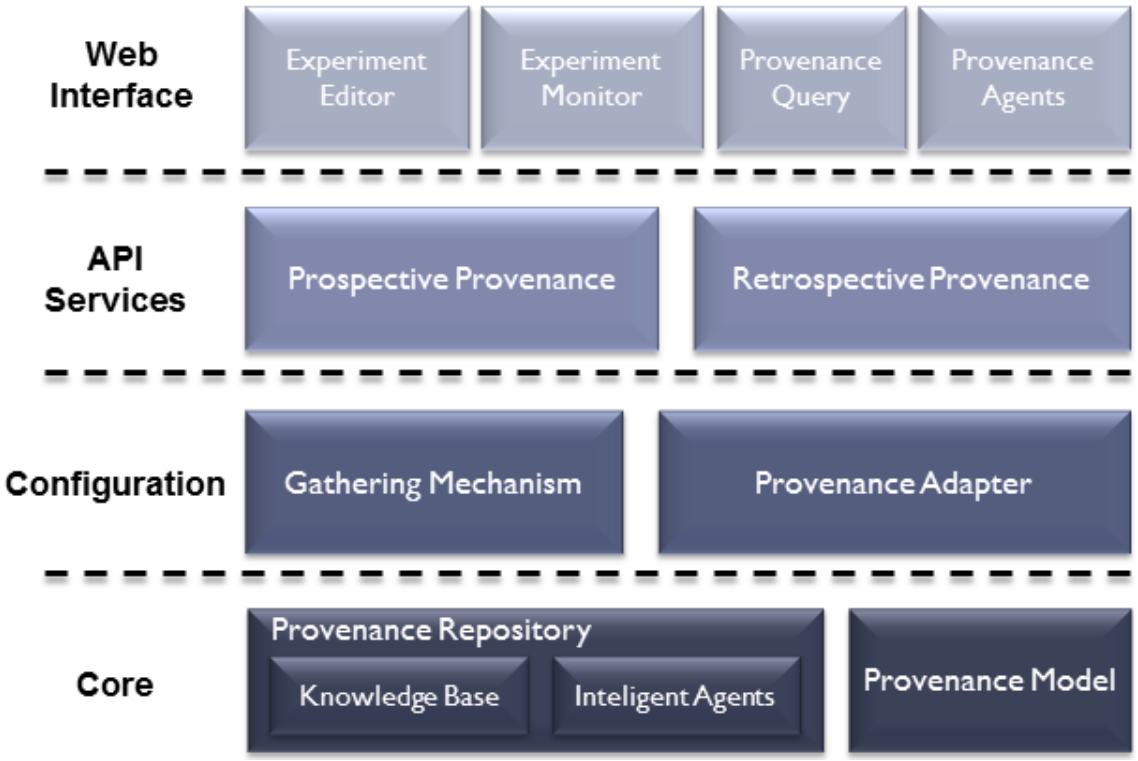


Figure 2. Architecture of ProvManager

The **Core layer** contains the base components for storing prospective and retrospective provenance data collected from the experiment. The *knowledge base* [14] and *intelligent agents* [15] interact with the provenance data to derive new information to the scientist by means of inference mechanisms.

The **Configuration layer** allows the gathering of provenance data at the activity level by configuring the workflow activities and using the component *Provenance adapter* for automating the task of adapting the activities.

The **API Services layer** allows publishing *Prospective* and *Retrospective* provenance data in ProvManager using Web services. Thus, provenance data can be published from remote machines according to the workflow activities distribution.

The **Web Interface layer** provides components that allow scientists to create and edit experiments; to monitor the execution of their experiments; to query the experiment provenance data; and to use intelligent agents in specific provenance analyses.

In the following sections, the main components are described in more detail.

3.1. Provenance model

The provenance model defined in ProvManager is based on the OPM. OPM represents the provenance information by a causality graph, which records the history of the workflow execution. This model is based on three main entities: Artifact, an immutable piece of state; Process, action or set of actions performed on or caused by artifacts that result in new artifacts; and Agent, contextual entity that acts on a process, enabling, facilitating, controlling and affecting its execution. Figure 3 illustrates these three entities and their possible forms of relationships, also called dependencies.

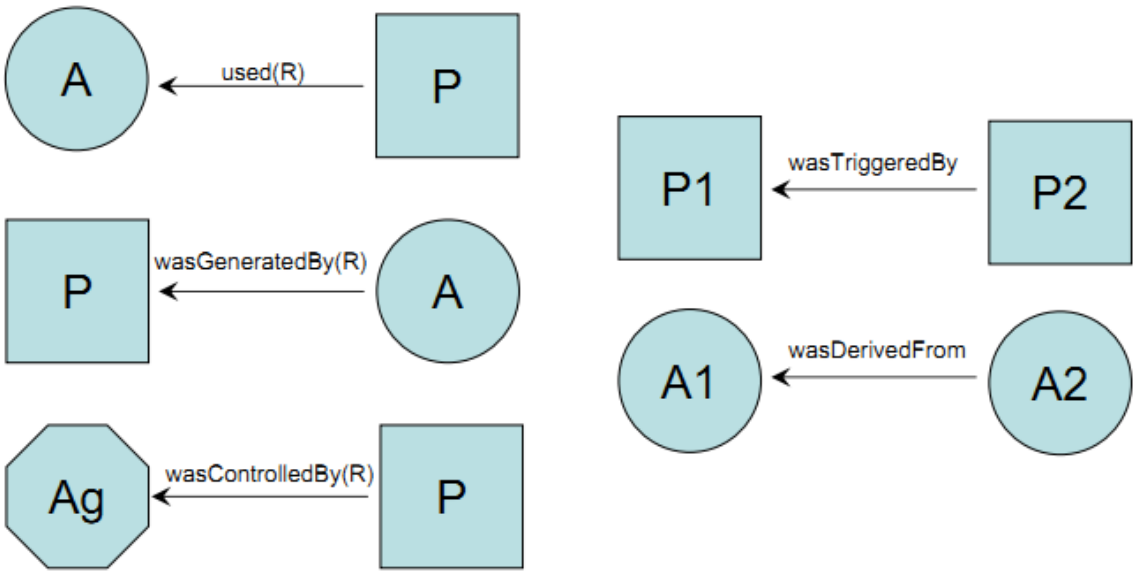


Figure 3. Entities and relationships in OPM (from [5])

In the left-hand side of Figure 3, the first two relationships express that a process *used* an artifact and an artifact that *was generated by* a process. These two dependencies represent derivation data relationships. The third relationship indicates that a process *was controlled by* an agent. Unlike the two aforementioned relationships, it represents a relationship of control. The fourth relationship is used in situations where no one knows exactly which artifacts were used by a process. Nevertheless, it is known that the process has used some artifact that was generated by another process. In this case, one can say that the process *was triggered by* another process. Similarly, the fifth

relationship is used in situations when the process that generated some artifact is unknown, but it is known that this artifact *was derived from* another artifact. These last two relationships are recursive and can be implemented by means of inference rules. Therefore, from them, it is possible to determine the sequence of processes that were executed in the workflow or the data derivation history from a specific artifact.

OPM is an important initiative focused on interoperability. Therefore, it supports a simple basic canonic model, which presents limitations for ProvManager goals. For instance, OPM lacks information about the workflow specification (prospective provenance), and thus it can be classified only as a retrospective provenance model. However, it is important that a provenance model combines the two perspectives of provenance to allow for a more comprehensive analysis. Figure 4 presents two scenarios where prospective and retrospective provenance data are adopted together to provide a better understanding of the experiment results. The prospective information is represented by a workflow specification (modeled as a UML activity diagram), whereas the retrospective information is represented by a simple execution log. The first scenario shows that an incorrect interpretation may occur if the retrospective information is analyzed without considering the prospective information. By doing this, we can incorrectly suppose that activity B precedes activity A. This is not true because the prospective information states they can start in parallel. In the second scenario, if we just look at the prospective information we do not know which activity was actually executed after the decision point (activity B or activity C?). This information can only be known after analyzing the retrospective information, which, in this case, indicates that activity B was the selected one. Moreover, OPM represents provenance as directed acyclic graph (DAG), which means that control-flow structures such as decision points and loops are not explicitly registered. It is important to note that DAG is suitable to represent retrospective provenance, which is the focus of OPM, but it is not appropriate to represent prospective provenance with complex control-flow structures.

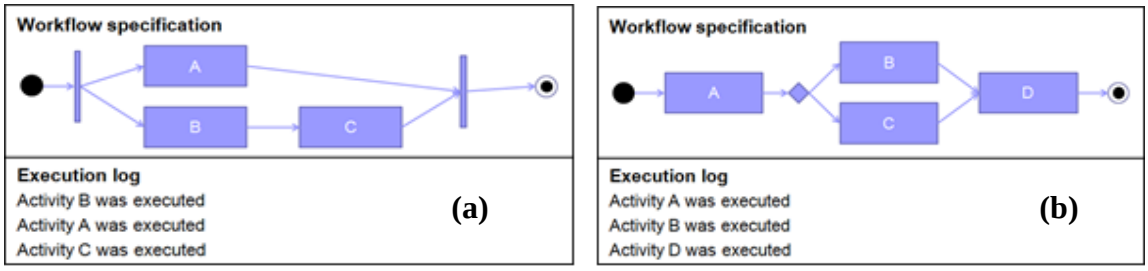


Figure 4. Two examples of prospective and retrospective provenance information used together to improve the provenance analysis: (a) prospective provenance clarifies the interdependence between activities; (b) retrospective provenance clarifies the alternatives of prospective provenance

Due to these previous considerations, the provenance model of ProvManager is based on OPM with some additional characteristics. The ProvManager's model deals with prospective provenance besides retrospective provenance. In order to enable a provenance model with prospective information, the ProvManager's model supports the representation of directed cyclic graph (DCG) workflows. For that reason, it supports the representation of workflow control-flow structures such as decision points, synchronism points, and other iteration-based structures necessary for representing

loops. Provenance models with DAG features like OPM cannot support some of these structures. Moreover, the provenance model of ProvManager was defined to support recursion, enabling the representation of workflow activities that are composed by themselves. This is possible because workflow activities are modeled in the provenance model in two phases: definition phase, where the characteristics of the workflow activities are defined such as name, input and output ports, etc.; and instantiation phase, where the defined workflow activities are instantiated in the workflow structure. This characteristic promotes the reuse of workflow activities. As such, after a workflow activity is defined, it can be instantiated several times in workflows. In summary, these characteristics are important to make ProvManager as generic as possible to support most kinds of SWfMS notations.

3.2. Provenance adapter

A provenance gathering mechanism that works at the activity level implies that each workflow activity should gather its own provenance data. However, some adjustments in the workflow are necessary to allow the workflow activities to capture the provenance data and publish them into ProvManager. For simplicity, ProvManager could delegate the task of adapting the workflow activities to the scientist. However, to reduce the burden of scientists in using the approach, ProvManager provides a component (*Provenance Adapter*) that executes this task semi automatically. During the configuration process, the *Provenance Adapter* is also responsible for collecting prospective provenance data from the experiment.

From the ProvManager perspective, the provenance mechanism configuration (phase A of Figure 1) consists of two tasks: (*i*) the provenance adapter instruments the workflow specification provided by the scientist with the provenance gathering mechanism; (*ii*) during the instrumentation, the provenance adapter registers the prospective provenance in ProvManager, enabling the new specification of the adapted workflow to be returned to the scientist for further execution in the SWfMS.

This configuration process raises some issues. The first one regards the workflow specification, which may differ according to the SWfMS being used. Each SWfMS has its own characteristics, such as workflow language specification, execution mechanism restrictions, etc. For that reason, the provenance configurator should be prepared to configure workflows from different SWfMS. Another issue is on how to adapt an activity to support the provenance mechanism. Once again, the answer depends on the SWfMS. Some SWfMS write the activity code in the workflow specification, others work with service calls or executables. Moreover, some applications used in the experiment cannot be adapted (at least not in a direct manner). Examples of such applications are: web services hosted in remote machines that cannot be reached, proprietary applications that do not provide source code, legacy applications, etc.

Therefore, the adaptation of workflow activities is a complex task, because it varies according to the SWfMS, and also has limitations depending on the type of application used in the experiment. In other words, it is hard to directly adapt the workflow activity. We have adopted a solution that is both independent from SWfMS implementation and from the application used in the experiment. Therefore, to cover a variety of SWfMS, the Provenance Adapter architecture is extensible, supporting the notation of different SWfMS. The provenance adapter component supports the connection of specific SWfMS adapters. These adapters have the duty of configuring a workflow that has been

developed in a specific SWfMS. Figure 5 illustrates the architecture of the provenance adapter. ProvManager can have, for example, adapters for VisTrails [16], Kepler [17], and Taverna [18].

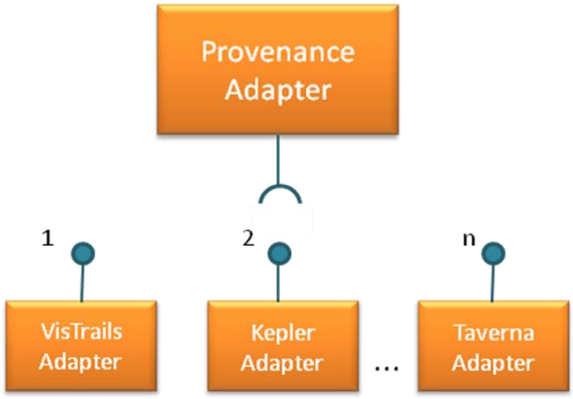


Figure 5. Architecture of the provenance adapter

Regarding the activity adaptation problem, the adaptation is done indirectly by modifying the workflow specification and inserting new workflow activities to collect the provenance information [12]. According to the workflow activity specification (e.g., output and input ports and how they are connected to other activities), Provenance Gathering Activities (PGA) are created to intercept data consumed or produced by each activity port. Additionally, information about the activity execution time is collected by other specific PGA. All this retrospective provenance information is published in ProvManager by PGA using the retrospective provenance API (see Section 3.4). Finally, the workflow region containing the original workflow activity and the related PGA are "wrapped" into a composite activity in order to maintain the original workflow visual aspect.

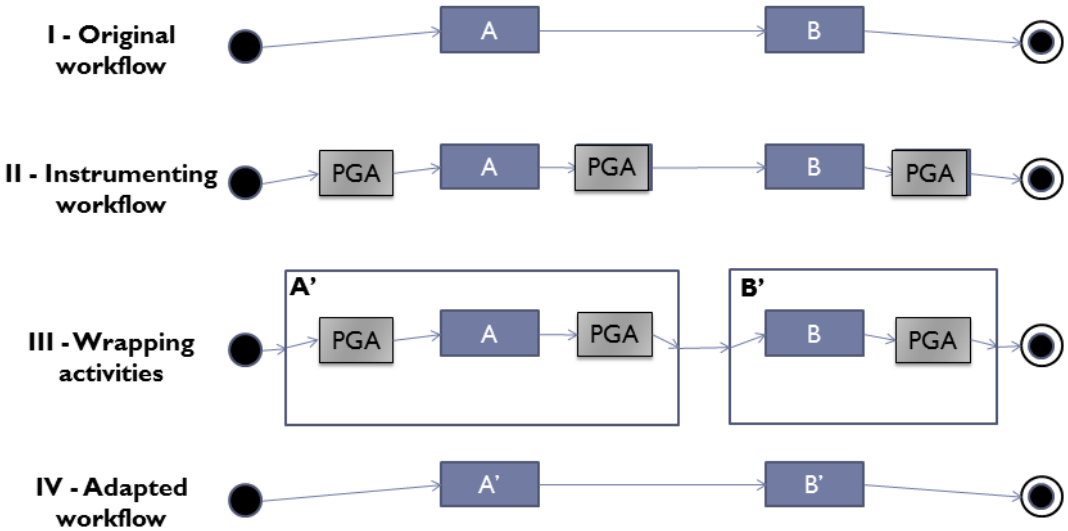


Figure 6. Wrapper structure of an adapted activity

Figure 6 summarizes the stages of the workflow instrumentation process: at stage I we have the original workflow; the workflow is instrumented with PGA at stage II; the workflow activities along with PGA are wrapped into composite activities at stage III;

and, finally, at stage IV, we have the high level view of the adapted workflow, which presents no modifications in the workflow structure compared to the original one (at stage I).

3.3. Provenance repository

The provenance repository component stores the provenance data gathered by ProvManager. This repository uses a knowledge base with an inference engine for storing and deriving new information from the provenance repository data. The provenance data derivation process is improved by combining prospective and retrospective provenance data. In some cases, the prospective provenance data stored in the repository can be used to derive retrospective provenance data. For instance, using the example of Figure 4.b, a simple execution ending notification issued by the workflow activity B can act as a trigger to derive the knowledge of which activities in the decision point have not been executed. In the case of the example, we can infer that workflow activity C was not selected to be executed. The use of inference mechanisms to derive provenance data is one of the advantages highlighted by OPM [5].

The provenance data is stored in the knowledge base as Prolog predicates. Prolog is adopted because it is a language that natively provides full support for inference. The inference engine is improved through intelligent agents. Agents can be constructed to manipulate the data in the knowledge base to execute specific provenance analysis. Section 3.5 describes these agents in more detail.

3.4. Provenance API

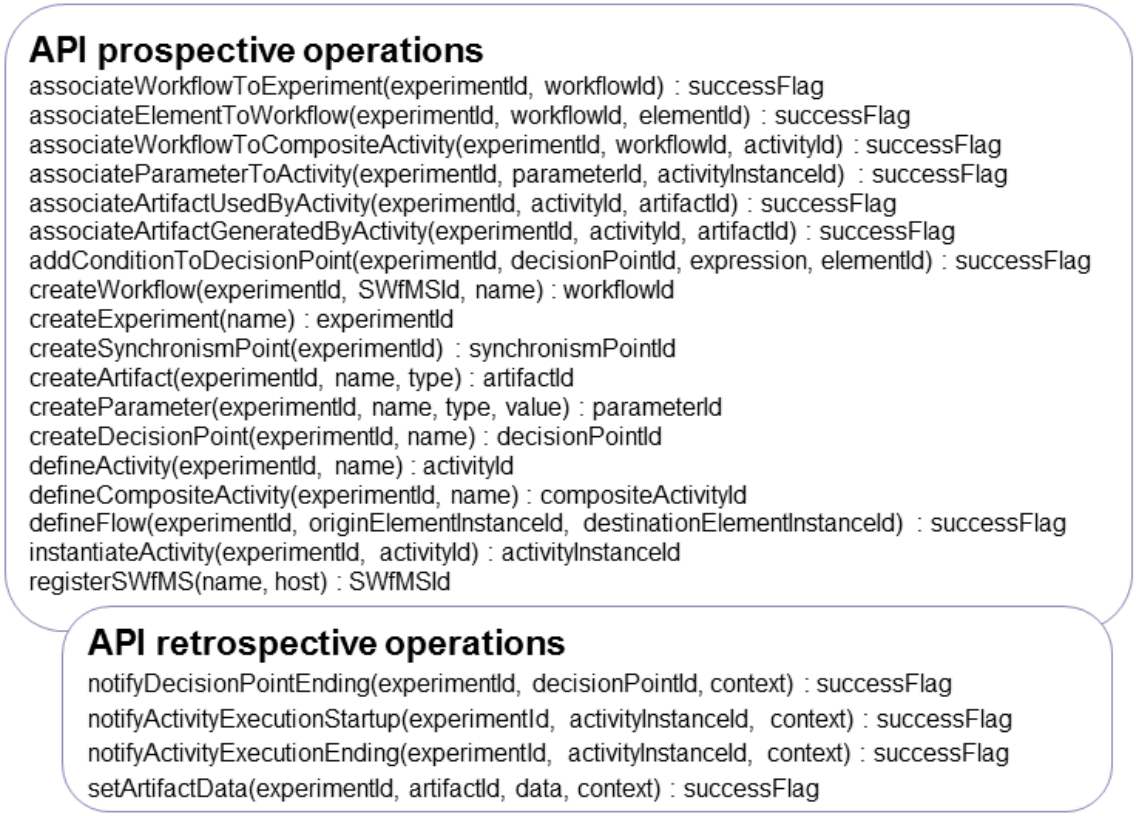


Figure 7. API provenance repository

The provenance repository provides a web services API for the communication within workflow activities, which can be executed at different SWfMS in different machines. This API helps to make the provenance gathering mechanism more independent of the provenance repository, since the first does not need to know how the provenance repository was implemented and vice versa. The service operation signatures are defined at a high abstraction level, encapsulating how data in the knowledge base are stored in Prolog predicates. However, internally, each operation maps its functionality into Prolog predicates in order to store the received data in the provenance repository.

Figure 7 shows the API provenance repository, in which the operations are divided into prospective and retrospective. The API integrates prospective and retrospective provenance information via some of the operations arguments, such as activities ids. For example, when a workflow activity is instantiated using the operation *instantiateActivity*, an *id* for the activity instance is created (*activityInstanceId)*. The same *id* is used to refer to that activity instance in situations such as when its execution has started (*notifyActivityExecutionStartup*) or finished (*notifyActivityExecutionEnding*).

3.5. Provenance agents

Queries on ProvManager repository are made using Prolog. The provenance agent component was created for helping the scientist in the provenance query and analysis. The term provenance agent is an adaptation of the term intelligent agent [15]. The provenance agent interacts with the provenance knowledge base to provide specific answers to the scientist according to its objectives and plans.

The adopted strategy is to create a variety of prefabricated provenance agents that deal with specific provenance applications. The agents can be configured to work adequately in particular scenarios. A strong point of this strategy is the low burden imposed to the scientists who do not need to construct agents. They just have to configure them. In the following, we list some provenance agents that are envisioned to help the scientist in the provenance analysis process.

Provenance Auditor Agent: This agent helps the scientist to run audit trails, identifying a derivation process (listing the applications and the manipulated data) that generated a specific result. In domains such as medicine and pharmacology, this audition is important when new drugs are discovered and the generation process should be verified in order to create patents [19]. Additionally, the provenance auditor agent can be used to verify the experiment resources usage. The agent can provide statistics of one or more resources, indicating whether it is being used or not. This kind of information is interesting to help the scientist to optimize the workflow.

Provenance Debugger Agent: Provenance information can help scientists to debug a workflow, since it contains information about the workflow execution. However, due to the large amount of provenance data, this task is very complex to be manually accomplished. The provenance debugger agent is conceived to ease this process. In order to debug a workflow, this agent works with both prospective and retrospective provenance information. The prospective provenance is used to obtain the workflow specification, whereas the retrospective information is used to obtain the workflow execution data. From this information, the debugger agent provides to the scientist activities or regions of the workflow that are not working adequately. In other words, it provides regions where the retrospective provenance does not match the prospective provenance.

Provenance Viewer Agent: This agent helps the scientist to retrieve and analyze information from the provenance repository. The agent is able to interpret a high level query language in order to simplify the query process to the scientist. Such queries are translated to Prolog and sent to the provenance repository. The provenance viewer agent is designed to work with provenance information filters, enabling the scientist to visualize the input and output data of a unique activity or complex regions that are composed of more than one activity. In literature, this functionality is known as user views [20].

Internally, the **Retrospective Provenance Agent (RPA)** controls all the provenance information sent, via web services API, by several activities during the experiment execution. RPA has to distinguish information from a specific experiment run (since the same experiment can be executed several times) and to optimize the retrospective provenance publishing by using inference about the stored prospective provenance information. For example, the adapted activity *GetData* from Figure 9 has a PGA that informs the end of its execution. When RPA receives this information, it can infer that activity *Validate* started its execution because this activity depends only of *GetData*. Therefore, another PGA informing that the activity Validate has started its execution is no more necessary. This contributes to decreasing the network overload due to a great number of PGA transmitting information to the provenance repository. The Prolog rule that represents this inference is shown the Figure 8.

```
finish(activity(IdA), C, T) :- !, executing(activity(IdA), C, Ti, Performers),
finished(IdA, C, Tf, _), Tf > Ti, Tf =< T, retract(executing(activity(IdA), C,
Ti, Performers)), assertz(executed(activity(IdA), C, Ti, Tf, Performers)),
findall(E, transition(activity(IdA), E), Es), start(Es, C, Tf))
```

Figure 8. Prolog rule to finish a workflow activity execution

The semantic of the rule is as follows. If the activity exists, is currently running, and has been finalized by some user, then change its state to *executed* in a certain context (C), with a specific finishing time (T), and try to start the execution of the activities that are at the destination flow of this activity.

4. PROVMANAGER IN ACTION

This section shows a small example of ProvManager in action. Figure 9.a illustrates the experiment structure that we use as an example. This experiment is segmented in two workflows: one workflow is instantiated in Kepler [17] that invokes parallel execution through Hadoop [21], while the other in VisTrails [16] focuses on visualization of the generated results. Figure 9.b shows the workflow in VisTrails with more details. The fragment is composed of three activities: *GetData*, *Validate*, and *Simulate*, running on a remote host with IP address 192.168.0.5. To capture provenance data from this experiment, the scientist has to publish it in ProvManager, uploading the workflow specifications (in the case of VisTrails, a .VT file). At this moment, ProvManager configures the workflow, by automatically adding PGA activities that are responsible for capturing and publishing retrospective provenance data in ProvManager during the workflow execution.

Currently, ProvManager can only instrument workflows executed in Kepler and VisTrails. However, as discussed before, ProvManager works with specific adapters to

be able to support future extensions to other SWfMS instrumentation mechanisms. Finally, at the end of the instrumentation, a new .VT file is returned to the scientist to be reloaded in VisTrails. This workflow specification is the one that should be executed. During the instrumentation and execution of the workflow, ProvManager captures provenance data from the workflow and publishes these data in the repository. This repository is a Prolog database, so provenance data are mapped into Prolog predicates. Figure 9.b shows the .VT file mapped into Prolog predicates.

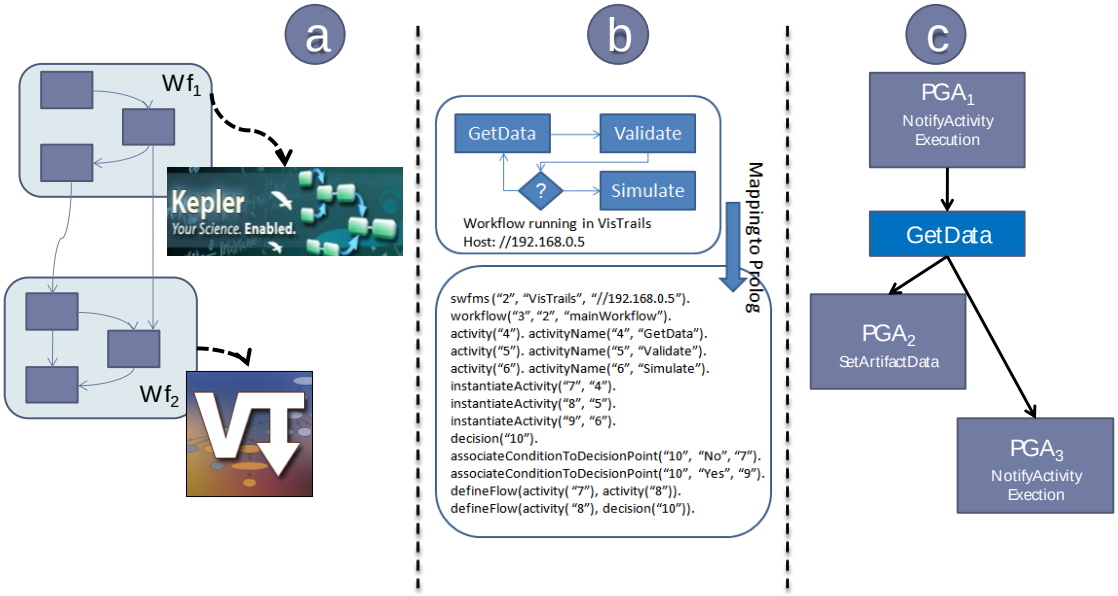


Figure 9. Experiment example (a), VisTrails workflow mapped to Prolog predicates (b), and workflow adaptation in VisTrails (c).

To register an experiment in ProvManager, first the scientist needs to create an experiment entity through the web interface. An experiment entity can be associated with one or more workflows. A workflow in ProvManager represents an experiment segment that is executed by a SWfMS. Workflows are created by the scientist according to the experiment structure. In the example of Figure 9.a, the workflows in VisTrails and Kepler represent segments of the same experiment.

To publish a workflow in ProvManager, the scientist should provide the workflow specification to be adapted. After publishing the workflow, the scientist can download the adapted specification in order to actually execute it in the SWfMS, as shown in Figure 10.a. Figure 9.c illustrates some operational details on how the workflow activity *Validate* is adapted using composite activities in VisTrails. A composite activity is created by using the VisTrails functionality called *Group*. Notice that some PGA is placed before the activity execution (PGA_1), and others are placed after it (PGA_2 and PGA_3). This decision depends on the type of provenance that needs to be gathered. For instance, the PGA agents that use the API operation *notifyActivityExecutionStart* have to be executed before the original activity in the sub-workflow. The opposite happens to PGA that uses the API operation *notifyActivityExecutionEnd*.

ProvManager provides a basic query mechanism to visualize the collected provenance data as illustrated in Figure 10.b. The scientist types a query expression in

the input text and the result is returned in the text area below. This mechanism also helps the scientist to type the expression by means of a suggestion mechanism that exhibits possible queries according to what the scientist is typing. These suggestions can be augmented with more expressions since the query mechanism allows scientist to save each query. Besides, the scientist may store high level query expressions. Finally, ProvManager provides a mechanism for monitoring the experiment execution. This mechanism is interesting since it provides a global view of the experiment execution to the scientist, saving his/her time in visiting each SWfMS to verify whether an experiment fragment has been executed or not. For example, Figure 10.c shows the monitoring of the VisTrails workflow fragment.

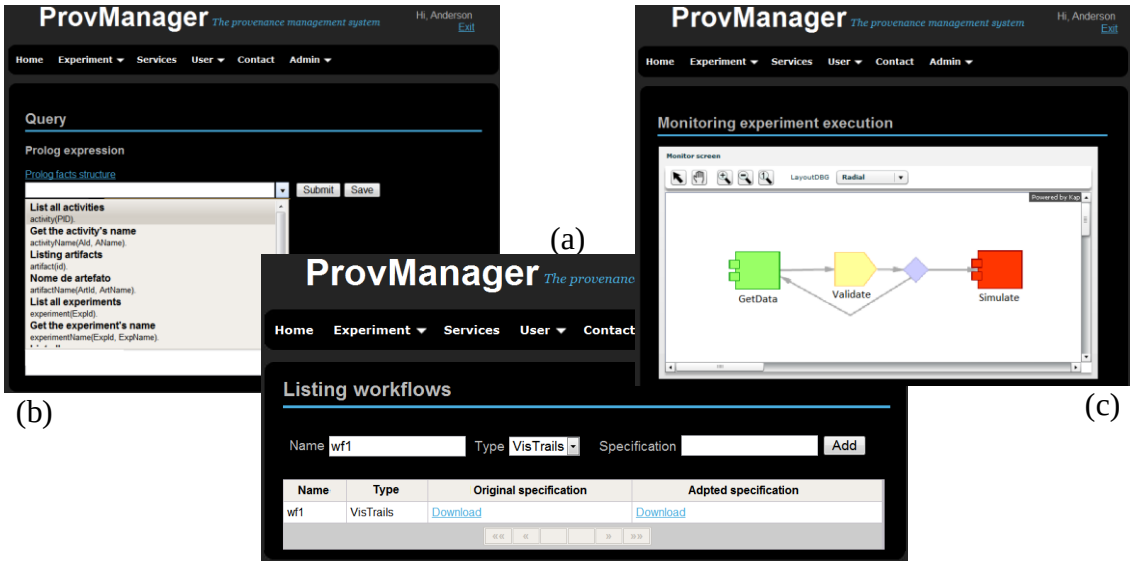


Figure 10. ProvManager screens: (a) List of workflows from one experiment; (b) Query interface; (c) Execution monitoring

5. EVALUATION

ProvManager was designed to support the provenance management of experiments in an integrated way, absorbing all information that is collected individually for each system in a distributed and heterogeneous environment. In this paper we evaluate if the scientist can take advantage of ProvManager to enhance the provenance analysis of an experiment that is executed in a complex heterogeneous environment. With this objective in mind, we planned an observational study to assess the performance and to obtain the evaluation of scientists using two different provenance analysis strategies: decentralized approach, in which scientists use the provenance mechanisms available in existing SWfMS where experiment workflow fragments are executed; and the ProvManager approach (or centralized approach), in which scientists only depend on the provenance information collected by ProvManager to make their analysis. The following sub-sections present the evaluation plan, its execution and the obtained results.

5.1. Planning

A hypothetical experiment, named *CHExp* (*Complex Heterogeneous Experiment*), was

defined to be performed in a heterogeneous environment. According to this scenario, this experiment is fragmented into two workflows, instantiated in two distinct SWfMS: VisTrails and Kepler. The conceptual model of *CHExp* experiment can be seen in the activity diagram of Figure 11. *CHExp* experiment consists of receiving a zipped file with parameters that should be unzipped, processed to extract its parameters that are finally used to calculate an average value. According to the diagram, part of the experiment workflow is performed in the Kepler workflow management system and the other part in the VisTrails system. Figure 12 presents the instantiated *CHExp* experiment in Kepler and VisTrails systems.

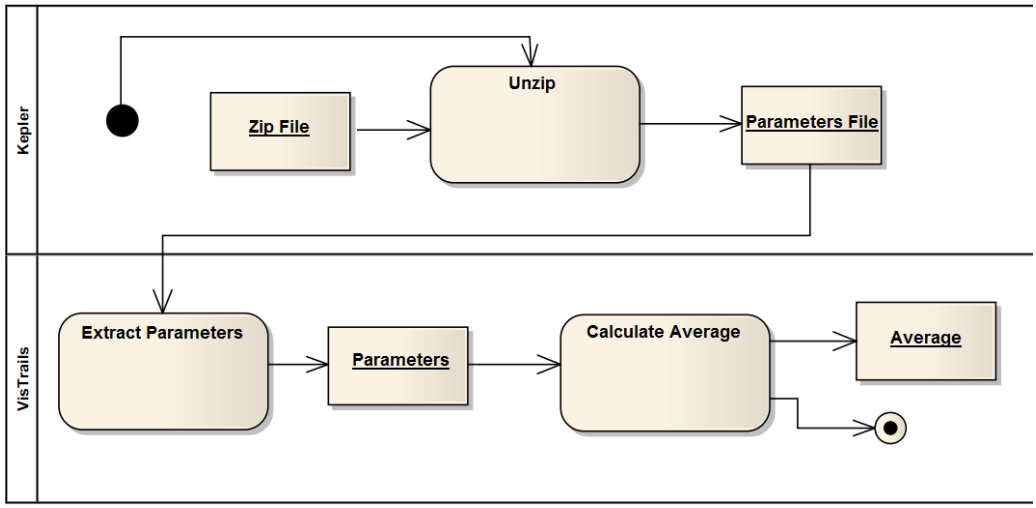


Figure 11. Diagram of *CHExp* experiment

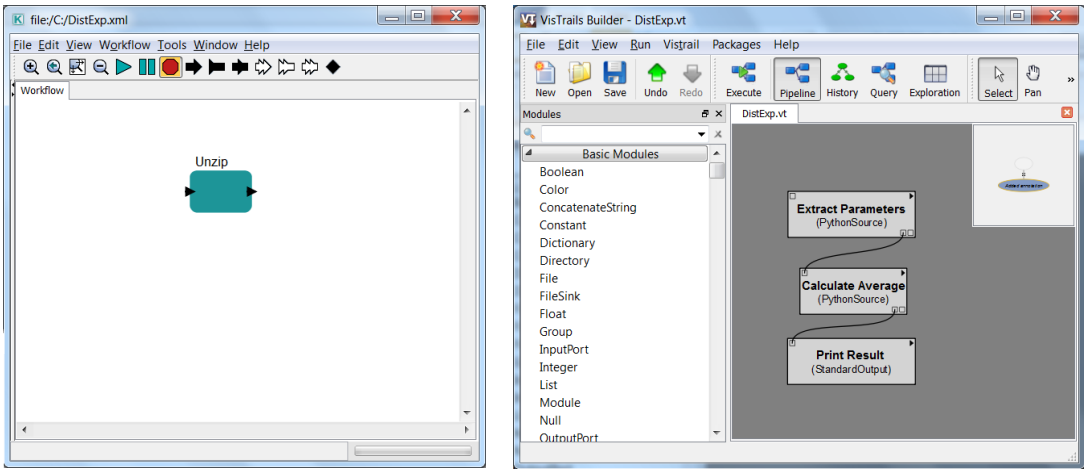


Figure 12. Instantiated *CHExp* experiment in *Kepler* (left window) and *VisTrails* (right window) systems

The study consisted in analyzing the performance of participants, which play the scientist role, in the provenance analysis process of *CHExp* experiment adopting each of the following strategies: using the set of existing provenance mechanisms of VisTrails and Kepler SWfMS (strategy A); and only using the ProvManager prototype (strategy B).

The provenance analysis process consists on providing the participant with a set of provenance questions referring to *CHExp* experiment to be answered. Among these questions, there is a subset of prospective provenance questions (*e.g.*, to describe the input and output ports of an activity) and a subset of retrospective provenance questions (*e.g.*, to identify the experiment or activity execution time). There are also other complex provenance questions that explore the analysis of the experiment as a whole, not just the workflow. For example, one of the questions asked the participants to describe the sequence of transformations of a certain information until its final state. This question can only be answered by analyzing the two experiment workflows.

The experiment was conducted in an academic environment, at the Software Engineering Lab at Federal University of Rio de Janeiro. There were five postgraduate students participating in the experiment. The participants' level of expertise in scientific workflow was very low. The two exceptions were one participant who had already used the Kepler system, however did not present too much experience, and another one who had read some material about the subject. Regarding the use of Prolog, the expertise grade of participants was also very low, with two participants who had no knowledge at all in the subject. This was interesting to simulate the knowledge grade of scientists of non technological areas.

5.2. Execution

Each session of the study used a unique participant and was divided into two phases. In each of them we observed the participant's behavior while answering the provenance questions for the two provenance analysis strategies. Participants were divided into two groups, where the ordering of the phases was different for each group. For one group (Group 1), the first phase of the session was to use strategy A, followed by the phase where strategy B was used. For the other group (Group 2), the sequence was inversed. Each session lasted for about 70 minutes.

At the beginning of the study session, the participant received a training of approximately 10 minutes about the main concepts of scientific experimentation. Apart from that, the participant also received a specific training for each phase, of approximately 10 minutes. During strategy A phase, the participant received a specific training about Kepler and VisTrails SWfMS, learning how to manipulate their main functionalities. During strategy B phase, the participant received a training related to the main concepts and functionalities of the ProvManager prototype. The participant also received a specific training on Prolog to allow him/her to build provenance queries in Prolog while using ProvManager.

It is worth mentioning that at each phase a provenance questions form similar to the one in Figure 13 was used, changing only execution parameters. This was meant not to ease the participant's task and avoid learning biases. They had half an hour to answer the provenance questions in each phase. There were 6 provenance questions. Figure 13 presents these provenance questions. Participants should try to answer most questions in each phase, not having to answer all of them. At the end of their task, participants had to answer an Evaluation Form, commenting on their experience of performing the tasks and on the use of the ProvManager prototype.

Prospective Provenance

1) Which ports does the activity "*ExtractParameters"* have?
2) Which activities does the VisTrails workflow have?

Retrospective Provenance

3) What was the execution time of the experiment "*CHExp*"?
4) What was the execution time of the workflow activity "*Unzip"*?
5) Which were the data generated and consumed by the workflow activity "*Unzip"*?
6) What is the derivation trace of the data generated by the workflow activity "*CalculateAverage*"?

Figure 13. Provenance questions form

5.3. Results

In general, participants' behavior was similar throughout the study. They all had more difficulty in using strategy A when compared to strategy B. While using strategy A, participants only succeeded answering prospective provenance questions. The most complex questions were the ones that needed information from the two SWfMS, as for example, the experiment execution time and the transformation traceability of an artifact. However, the participants had no difficulty at all on answering the remaining prospective provenance questions. The reason for that being that the provenance information was extracted from the functionalities provided by graphical resources, which allowed participants to visualize the workflow structure. However, it is worth noting that the used functionalities were not specific to provenance, but to the workflow editing mechanism.

By using strategy B, participants were able to answer all provenance questions, both prospective and retrospective. Differently from strategy A, participants easily answered questions that dealt with the analysis of the two experiment workflows. However, participants had a little bit of difficulty while building Prolog queries. Queries were built manually by participants, without the aid of auto-completing or correction suggestions mechanisms for incorrectly written queries. Thus, whenever the participants made a typing error, they thought that they were not being able to correctly deal with Prolog predicates while using ProvManager. In this situation, the researcher in charge pointed out the typing error to the participant. Apart from that, participants did not find any other difficulty to answer the questions. Table 5.3 summarizes the results of the participants in answering the provenance questions according to each strategy. A right question is represented by the symbol (✔), while a question answered incorrectly, or not answered, is represented by the symbol (✘).

In which concerns evaluation form participants found easier to use the ProvManager approach than the combined Vistrails and Kepler approach. They thought that the ideal solution is a centralized one, in comparison to a decentralized use of various provenance systems with different interfaces and distinct provenance models. Finally, it is worth mentioning that this approach was presented to professionals of industry that work with scientific workflow. They considered the approach very promising and shown the interest of using ProvManager in their works.

Table 1. Results of the participants in answering the provenance questions

Provenance form			Group 1			Group 2	
			P1	P2	P3	P4	P5
VisTrails + Kepler	Prospective	Question 1	✓	✓	✓	✓	✓
		Question 2	✓	✓	✓	✓	✓
	Retrospective	Question 3	✗	✗	✗	✗	✗
		Question 4	✗	✗	✗	✗	✗
		Question 5	✗	✗	✗	✗	✗
		Question 6	✗	✗	✗	✗	✗
ProvManager	Prospective	Question 1	✓	✓	✓	✓	✓
		Question 2	✓	✓	✓	✓	✓
	Retrospective	Question 3	✓	✓	✓	✓	✓
		Question 4	✓	✓	✓	✓	✓
		Question 5	✓	✓	✓	✓	✓
		Question 6	✓	✓	✓	✓	✓

6. RELATED WORK

The ProvManager architecture is a step towards helping scientists to diminish their efforts on configuring a provenance gathering mechanism in a distributed and heterogeneous environment scenario, where several workflows can be used to compose a single experiment. There are several approaches to manage provenance of scientific workflows, such as: Karma [7], PASOA [8], Wings/Pegasus [22], Swift [23], VisTrails [16], Kepler [17] and Matriohska [9][24], among others. In this section we compare the characteristics of major provenance systems and approaches regarding the key characteristics in provenance management (Table 2).

Simmhan, Plale and Gannon [7] defined a framework named Karma for collecting provenance in data-centric scientific workflows. Karma collects provenance at workflow and operating system levels and the information is sent to a central repository.

Groth et al. [8] designed a provenance system named PASOA. This is one of the first work that discusses the advantages of separating the provenance gathering mechanism from SWfMS and suggests the use of OPM [5]. The system has its own model for provenance storage and operates at workflow and activity levels. It was designed to provide a service-oriented architecture to operate on grid environments.

Table 2. Comparisons between provenance systems adapted from [13]

	Provenance System	Capture Level	Capture Mechanism	Type of Provenance	Level of Coupling	Effort to Adapt the Workflow	Workflows Automatic configuration
Distributed Systems	Karma	Workflow & OS	External	Retrospective	Loose	High	No
	Matriohska	Workflow & OS	Internal & External	Retrospective Prospective	Loose	Medium	No
	PASOA	Workflow Activity	External	Retrospective	Loose	High	Yes
	Pegasus	Workflow	External	Retrospective	High	High	No
	ProvManager	**Activity**	**External**	**Retrospective Prospective**	**Loose**	**Low**	**Yes**
	Swift	Workflow	Internal	Retrospective	High	High	No
	View	Workflow	External	Retrospective	High	High	No
Centralized Systems	Kepler	Workflow	Internal	Retrospective	High	Low	No
	Taverna	Workflow	Internal	Retrospective	High	Low	No
	VisTrails	Workflow	Internal	Retrospective Prospective	High	Low	No

Matriohska [9] was initially conceived to operate on clusters and grid environments and to be decoupled from SWfMS, therefore, using native services from those environments, for example, process schedulers and queue managers. Matrioshka was later refactored to operate on cloud environments and to collect provenance of scientific experiments at workflow and operating system levels [24].

The Wings/Pegasus framework [22] implements a distributed provenance gathering system. Wings implements the first two layers of workflow creation, from workflow templates to workflow instances, while the Pegasus system transforms a workflow instance into an executable workflow through a mapping that assigns workflow tasks to available grid resources for execution. Pegasus maps and restructures the workflow to make its execution efficient by creating provenance information that relates the final executed workflow to the original workflow specification.

Swift [23] is a parallel programming system that integrates some elements to meet the scientist's needs. Swift's provenance-recording component is named Kickstart. It captures execution details for diagnosis and records them in a provenance database.

Different from the previous works, VisTrails is a centralized open-source SWfMS and provenance management system that provides support for data exploration and visualization [16]. VisTrails deals with prospective and retrospective provenance that is persisted as XML files or relational databases. VisTrails only collects provenance at the level of the workflow engine.

Another centralized workflow system is the Kepler system. Kepler is a Java-based SWfMS that is ported for different operating systems. Despite of being a mature system, Kepler provenance gathering mechanism is solely centered at workflow level and is still under development.

As far as we are concerned, PASOA is the closest approach to ProvManager by gathering provenance at activity level, offering a loosely coupled approach to SWfMS and providing for automatic workflow configuration. However, from the public available information on PASOA, it does not combine prospective and retrospective

provenance data, and the automatic workflow configuration depends on how the applications were developed. The applications should be developed according to Prime methodology [10]. Besides, while PASOA is focused on the provenance of SOA applications, ProvManager works not only with SOA applications, but also with legacy scientific applications. Finally, further provenance analysis may be achieved with ProvManager by reasoning through the inference engine.

7. CONCLUSION

This paper presented an approach for managing provenance in scientific experiments. This approach, named ProvManager, is independent of SWfMS and was conceived to enable the management of provenance in different scenarios of workflow execution in a distributed and heterogeneous environment, where more than one SWfMS is used to run a single scientific experiment. Additionally, our approach aims at easing the work of the scientist regarding provenance gathering and analysis.

The strategy of working with a provenance gathering mechanism at the activity level can cause an overhead on workflow processing, since new activities (PGA) are inserted. In addition, depending on the gathered information, these PGA may also consume high network bandwidth, since they have to transfer all generated data from the workflow activities to the provenance repository. An alternative to address this limitation is to store data locally and publish only a pointer to the data at the repository, as suggested by Groth et al [8].

ProvManager is currently being tested. We have made an evaluation of ProvManager on the provenance analysis of a fictitious experiment that was executed in a heterogeneous environment. This evaluation presented some evidences that our approach is more efficient in the provenance analysis when compared to other provenance gathering strategies. As future work, we plan to evaluate our approach in real scenarios, using bioinformatics and oil engineering experiments that are being developed within our research group [25]. This evaluation will bring us feedback for future improvements. Furthermore, we are working on a more user-friendly interface to help scientists in the provenance querying process. Currently, it is done manually by typing the Prolog query statement. However, we are doing some research in this area [26], where we intend to adopt provenance query patterns to guide users while composing their queries.

ACKNOWLEDGEMENTS

This work is partially sponsored by CNPq, CAPES, and FAPERJ.

REFERENCES

[1] D. Hollingsworth, “Workflow Management Coalition: The Workflow Reference Model”, *The Workflow Management Coalition*, 1995.

[2] A. Goderis, U. Sattler, P. Lord, e C. Goble, “Seven Bottlenecks to Workflow Reuse and Repurposing”, *The Semantic Web – ISWC 2005*, Galway, Ireland: 2005, p. 323-337.

[3] J. Freire, D. Koop, E. Santos, e C.T. Silva, “Provenance for Computational Tasks: A Survey”, *Computing in Science and Engineering, v.10*, 2008, p. 11-21.

[4] Y.L. Simmhan, B. Plale, e D. Gannon, *A Survey of Data Provenance Techniques*, Computer Science Department, Indiana University, 2005.

[5] L. Moreau, J. Freire, J. Futrelle, R. McGrath, J. Myers, e P. Paulson, "The Open Provenance Model: An Overview", *Provenance and Annotation of Data and Processes*, 2008, p. 323-326.
[6] S. da Cruz, F. Chirigati, R. Dahis, M. Campos, e M. Mattoso, "Using Explicit Control Processes in Distributed Workflows to Gather Provenance", *Provenance and Annotation of Data and Processes*, 2008, p. 186-199.
[7] Y.L. Simmhan, B. Plale, e D. Gannon, "A Framework for Collecting Provenance in Data-Centric Scientific Workflows", *ICWS*, 2006, p. 427-436.
[8] P. Groth, S. Jiang, S. Miles, S. Munroe, V. Tan, S. Tsasakou, e L. Moreau, *An Architecture for Provenance Systems*, http://eprints.ecs.soton.ac.uk/13216/, 2006.
[9] S.M.S. da Cruz, P.M. Barros, P.M. Bisch, M.L.M. Campos, e M. Mattoso, "Provenance Services for Distributed Workflows", *Proceedings of the 2008 Eighth IEEE International Symposium on Cluster Computing and the Grid*, IEEE Computer Society, 2008, p. 526-533.
[10] S. Munroe, S. Miles, L. Moreau, e J. Vázquez-Salceda, "PrIMe: a software engineering methodology for developing provenance-aware applications", *Proceedings of the 6th international workshop on Software engineering and middleware*, Portland, USA: ACM Press New York, NY, USA, 2006, p. 39-46.
[11] C. Lin, S. Lu, Z. Lai, A. Chebotko, X. Fei, J. Hua, e F. Fotouhi, "Service-Oriented Architecture for VIEW: A Visual Scientific Workflow Management System", *Services*, IEEE Computer Society, 2008, p. 335-342.
[12] A. Marinho, L. Murta, C. Werner, V. Braganholo, S.M.S. da Cruz, e M. Mattoso, "A Strategy for Provenance Gathering in Distributed Scientific Workflows", *IEEE International Workshop on Scientific Workflows*, Los Angeles, California, United States: 2009.
[13] S.M.S. da Cruz, M. Campos, e M. Mattoso, "Towards a Taxonomy of Provenance in Scientific Workflow Management Systems", *IEEE International Workshop on Scientific Workflows*, Los Angeles, California, United States: 2009.
[14] A.J. Gonzalez e D.D. Dankel, *The Engineering of Knowledge-Based Systems*, Prentice Hall, 1993.
[15] S. Russell e P. Norvig, *Artificial Intelligence*, Prentice Hall, 2009.
[16] S.P. Callahan, J. Freire, E. Santos, C.E. Scheidegger, C.T. Silva, e H.T. Vo, "VisTrails: visualization meets data management", *SIGMOD*, Chicago, Illinois, USA: ACM, 2006, p. 745-747.
[17] I. Altintas, C. Berkley, E. Jaeger, M. Jones, B. Ludascher, e S. Mock, "Kepler: an extensible system for design and execution of scientific workflows", *Scientific and Statistical Database Management*, Greece: 2004, p. 423-424.
[18] D. Hull, K. Wolstencroft, R. Stevens, C. Goble, M.R. Pocock, P. Li, e T. Oinn, "Taverna: a tool for building and running workflows of services", *Nucleic Acids Research*, vol. 34, 2006, p. 729-732.
[19] S. Miles, P. Groth, M. Branco, e L. Moreau, "The Requirements of Using Provenance in e-Science Experiments", *Journal of Grid Computing*, vol. 5, mar. 2007, p. 1-25.
[20] S. Cohen, S. Cohen-Boulakia, e S. Davidson, "Towards a Model of Provenance and User Views in Scientific Workflows", *Data Integration in the Life Sciences*, Springer, 2006, p. 264-279.
[21] J. Wang, D. Crawl, e I. Altintas, "Kepler + Hadoop: a general architecture facilitating data-intensive applications in scientific workflow systems", *Proc. of 4th Workshop on Workflows in Support of Large-Scale Science*, Portland, Oregon: ACM, 2009, p. 1-8.
[22] J. Kim, E. Deelman, Y. Gil, G. Mehta, e V. Ratnakar, "Provenance trails in the Wings-Pegasus system", *Concurrency and Computation: Practice & Experience*, vol. 20, abr. 2008, p. 587–597.
[23] Y. Zhao, M. Hategan, B. Clifford, I. Foster, G. von Laszewski, V. Nefedova, I. Raicu, T. Stef-Praun, e M. Wilde, "Swift: Fast, Reliable, Loosely Coupled Parallel Computation", *Proc. of the 3rd IEEE World Congress on Services*, Salt Lake City, USA: 2007, p. 206, 199.
[24] S.M.S. Cruz, C.E. Paulino, D. Oliveira, M.L.M. Campos, e M. Mattoso, "Capturing Distributed Provenance Metadata from Cloud-Based Scientific Workflows", *Journal of Information and Data Management*, vol. 2, 2011, p. 43-50.
[25] GExp, *Brazilian project for supporting large scale management of scientific experiments*, http://gexp.nacad.ufrj.br/, 2009.
[26] L. Gadelha, M. Mattoso, M. Wilde, e I. Foster, "Provenance Query Patterns for Many-Task Scientific Computing", *USENIX Workshop on the Theory and Practice of Provenance*, Heraklion, Crete, Greece: 2011.